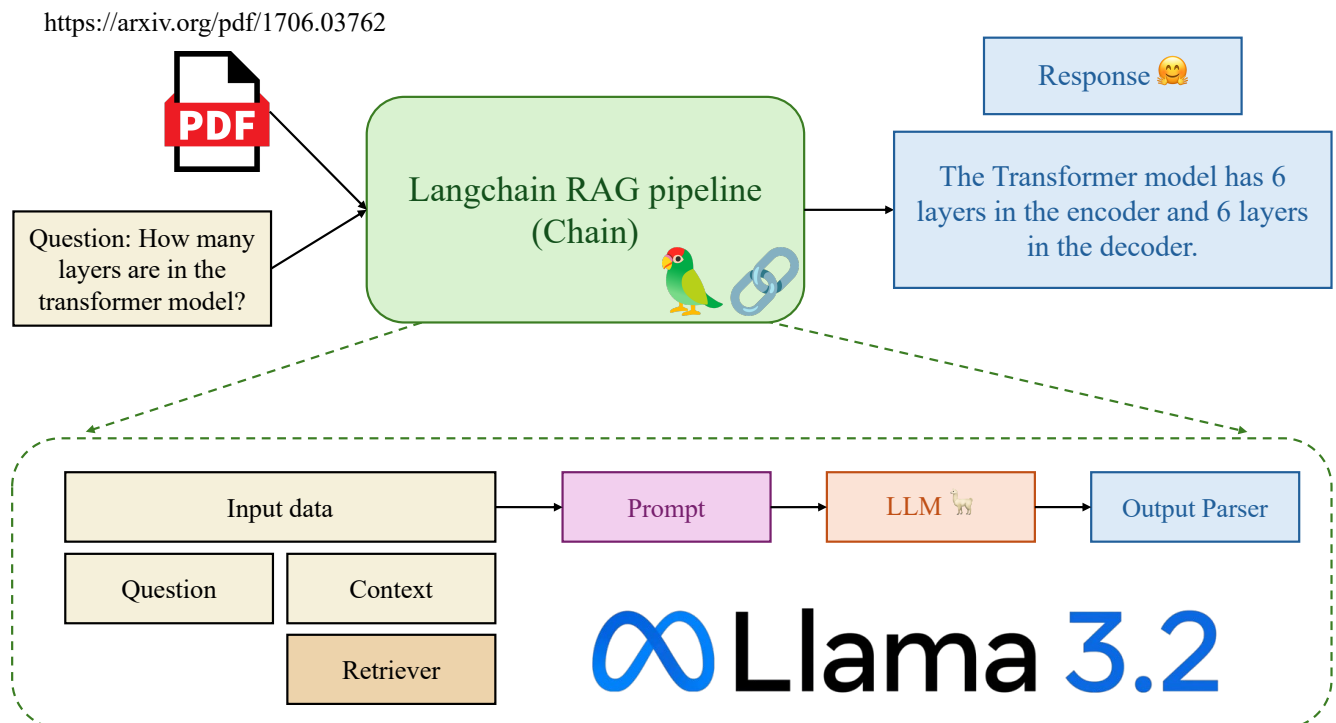


Tutorial: RAG with LangChain for PDF Question Answering

Dinh-Thang Duong, Nguyen-Thuan Duong, Quang-Vinh Dinh

I. Giới thiệu

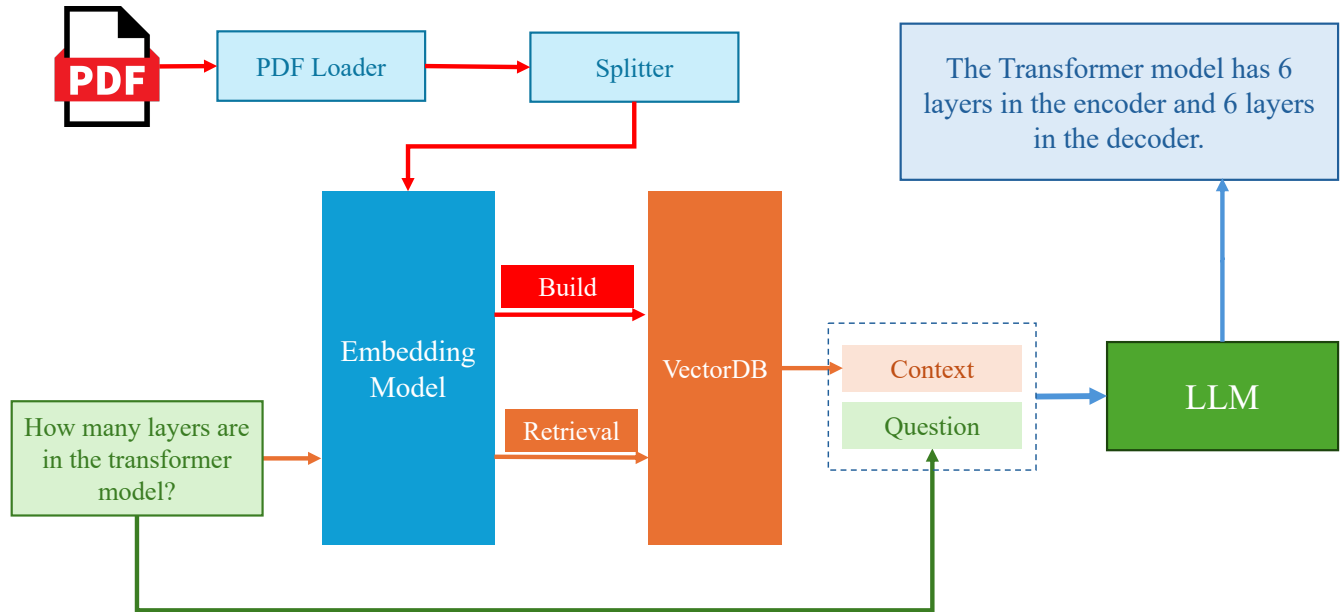
LangChain là một framework được thiết kế chuyên biệt cho việc triển khai LLMs trong các ứng dụng thực tế. LangChain hỗ trợ các công cụ và thư viện mạnh mẽ cho phép các nhà phát triển dễ dàng tích hợp các mô hình ngôn ngữ lớn với các ứng dụng của họ, từ các Chatbot thông minh cho đến các hệ thống phân tích dữ liệu phức tạp.



Hình 1: Minh họa ứng dụng hỏi đáp nội dung file pdf sử dụng LangChain.

Trong bài viết này, chúng ta sẽ xây dựng một ứng dụng về RAG (Retrieval Augmented Generation) trả lời các câu hỏi học thuật tận dụng nguồn tài liệu là các bài báo khoa học mà ta thu

thập được (dưới dạng file pdf), sử dụng thư viện LangChain. Tổng quan, pipeline của project như sau:



Hình 2: Tổng quan về pipeline của project.

Theo đó:

1. Từ danh sách các bài báo khoa học, ta tách thành các văn bản nhỏ. Từ đó, xây dựng một hệ cơ sở dữ liệu vector với một embedding model.
2. Bên cạnh câu hỏi đầu vào (question), ta truy vấn các mẫu văn bản có liên quan đến câu hỏi, dùng làm ngữ cảnh (context) trong câu prompt. Đây là nguồn thông tin mà LLMs có thể dựa vào để trả lời câu hỏi.
3. Đưa câu prompt vào mô hình (question và context) để nhận câu trả lời từ mô hình.

II. Cài đặt chương trình

Trong phần này, chúng ta sẽ tiến hành cài đặt nội dung của project. Mã nguồn được xây dựng trên hệ điều hành Ubuntu với GPU 24GB. Các bước thực hiện như sau:

II.1. Tổ chức thư mục code

Để mã nguồn trở nên rõ ràng nhằm phục vụ cho mục đích đọc hiểu code, chúng ta sẽ tổ chức thư mục như sau:

```
rag_langchain/
├── data_source/
│   └── generative_ai/
│       └── download.py
├── src/
│   ├── base/
│   │   └── llm_model.py
│   ├── rag/
│   │   ├── file_loader.py
│   │   ├── main.py
│   │   ├── offline_rag.py
│   │   ├── utils.py
│   │   └── vectorstore.py
│   └── app.py
└── requirements.txt
```

Tổng quan, chúng ta sẽ có thư mục chứa mã nguồn có tên **rag_langchain** (các bạn hoàn toàn có thể sử dụng tên gọi khác). Bên trong sẽ có các thư mục con và các file với ý nghĩa như sau:

- **data_source/**: Thư mục dùng để lưu trữ các tài liệu phục vụ cho việc xây dựng hệ cơ sở dữ liệu vector.
- **data_source/generative_ai/download.py**: File code dùng để tải tự động một số các bài báo khoa học dưới dạng file pdf.
- **src/base/llm_model**: File code dùng để khai báo hàm khởi tạo mô hình ngôn ngữ lớn.

_ **src/rag/**: Thư mục dùng để lưu trữ các code liên quan đến xây dựng RAG, bao gồm:

1. **src/rag/file_loader.py**: File code dùng để khai báo các hàm load file pdf (vì tài liệu của chúng ta thu thập thuộc file pdf).
2. **src/rag/main.py**: File code dùng để khai báo hàm khởi tạo chains.
3. **src/rag/offline_rag.py**: File code dùng để khai báo PromptTemplate.
4. **src/rag/utils.py**: File code dùng để khai báo hàm tách câu trả lời từ model.
5. **src/rag/vectorstore.py**: File code dùng để khai báo hàm khởi tạo hệ cơ sở dữ liệu vector.

_ **src/app.py**: File code dùng để khởi tạo API.

_ **requirements.txt**: File code dùng để khai báo các thư viện cần thiết để sử dụng source code.

II.2. Cập nhật file requirements.txt

Để bắt đầu, chúng ta sẽ liệt kê các gói thư viện cần thiết để chạy được chương trình này. Các bạn hãy cập nhật file requirements.txt với nội dung sau:

```

1 torch==2.2.2
2 transformers==4.39.3
3 accelerate==0.28.0
4 bitsandbytes==0.42.0
5 huggingface-hub==0.22.2
6 langchain==0.1.14
7 langchain-core==0.1.43
8 langchain-community==0.0.31
9 pypdf==4.2.0
10 sentence-transformers==2.6.1
11 beautifulsoup4==4.12.3
12 langserve[all]
13 chromadb==0.4.24
14 langchain-chroma==0.1.0
15 faiss-cpu==1.8.0
16 rapidocr-onnxruntime==1.3.16
17 unstructured==0.13.2
18 fastapi==0.110.1
19 uvicorn==0.29.0

```

II.3. Cập nhật file data_source/generative_ai/download.py

Để tải một vài các bài báo khoa học làm dữ liệu cho hệ cơ sở dữ liệu vector, chúng ta sẽ xây dựng một đoạn code tải tự động các bài báo. Nội dung như sau:

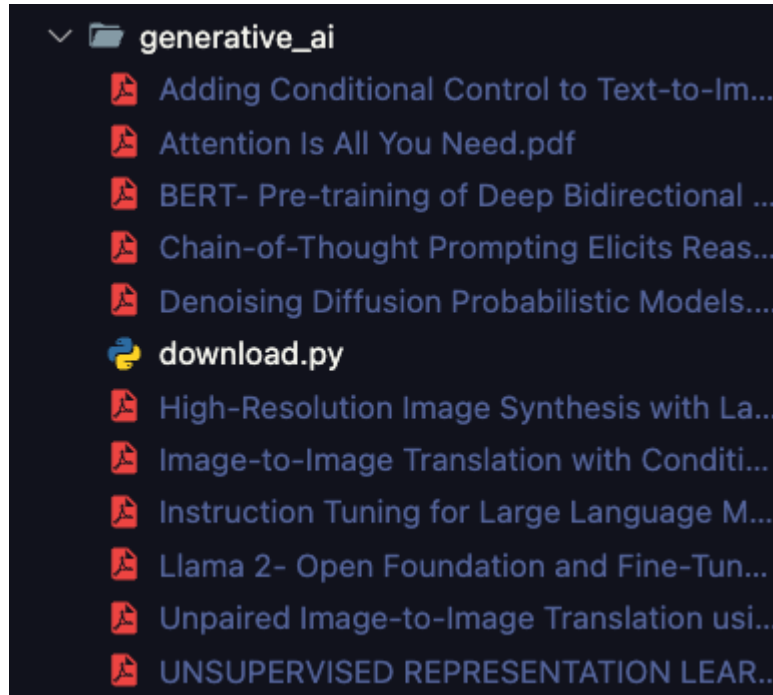
```

1 import os
2 import wget

```

```
3
4 file_links = [
5     {
6         "title": "Attention Is All You Need",
7         "url": "https://arxiv.org/pdf/1706.03762"
8     },
9     {
10        "title": "BERT- Pre-training of Deep Bidirectional Transformers for Language
11                Understanding",
12        "url": "https://arxiv.org/pdf/1810.04805"
13    },
14    {
15        "title": "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models",
16        "url": "https://arxiv.org/pdf/2201.11903"
17    },
18    {
19        "title": "Denoising Diffusion Probabilistic Models",
20        "url": "https://arxiv.org/pdf/2006.11239"
21    },
22    {
23        "title": "Instruction Tuning for Large Language Models- A Survey",
24        "url": "https://arxiv.org/pdf/2308.10792"
25    },
26    {
27        "title": "Llama 2- Open Foundation and Fine-Tuned Chat Models",
28        "url": "https://arxiv.org/pdf/2307.09288"
29    }
30 ]
31 def is_exist(file_link):
32     return os.path.exists(f"./{file_link['title']}.pdf")
33
34 for file_link in file_links:
35     if not is_exist(file_link):
36         wget.download(file_link["url"], out=f"./{file_link['title']}.pdf")
```

Trong file code trên, chúng ta cung cấp một list các đường dẫn bài báo. Từ đó, sử dụng `wget` để tải về. Các bài báo sẽ được lưu ngay tại vị trí của file code. Vì mục đích demo, chúng ta sẽ chỉ tải một số lượng nhỏ các paper. Các bạn có thể tự thêm vào nhiều paper khác để test.



Hình 3: Minh họa danh sách các file bài báo khoa học sau khi được tải về.

II.4. Cập nhật file `src/base/llm_model.py`

Tại file này, ta khai báo hàm `get_hf_model()`, dùng để thực hiện tải và gọi pre-trained LLM từ HuggingFace về máy. Đồng thời, ta áp dụng kỹ thuật quantization lên model để thực hiện inference trên GPU thấp. Nội dung file như sau:

```

1 import torch
2 from transformers import BitsAndBytesConfig
3 from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline
4 from langchain.llms.huggingface_pipeline import HuggingFacePipeline
5
6
7 nf4_config = BitsAndBytesConfig(
8     load_in_4bit=True,
9     bnb_4bit_quant_type="nf4",
10    bnb_4bit_use_double_quant=True,
11    bnb_4bit_compute_dtype=torch.bfloat16
12 )
13
14 def get_hf_llm(model_name: str = "meta-llama/Llama-3.2-3B-Instruct",
15               max_new_token = 1024,
16               **kwargs):
17
18     model = AutoModelForCausalLM.from_pretrained(
19         model_name,
20         quantization_config=nf4_config,
21         low_cpu_mem_usage=True
22     )
23     tokenizer = AutoTokenizer.from_pretrained(model_name)

```

```

24
25 model_pipeline = pipeline(
26     "text-generation",
27     model=model,
28     tokenizer=tokenizer,
29     max_new_tokens=max_new_token,
30     pad_token_id=tokenizer.eos_token_id,
31     device_map="auto"
32 )
33
34 llm = HuggingFacePipeline(
35     pipeline=model_pipeline,
36     model_kwargs=kwargs
37 )
38
39 return llm

```

Trong project này, mô hình LLM mà chúng ta sử dụng là mô hình [Llama 3.2 3B](#) được huấn luyện trên dữ liệu instruction. Các bạn có thể thay thế bằng mô hình khác có cấu hình tương tự.

II.5. Cập nhật file `src/rag/file_loader.py`

```

1 from typing import Union, List, Literal
2 import glob
3 from tqdm import tqdm
4 import multiprocessing
5 from langchain_community.document_loaders import PyPDFLoader
6 from langchain_text_splitters import RecursiveCharacterTextSplitter
7
8 def remove_non_utf8_characters(text):
9     return ''.join(char for char in text if ord(char) < 128)
10
11 def load_pdf(pdf_file):
12     docs = PyPDFLoader(pdf_file, extract_images=True).load()
13     for doc in docs:
14         doc.page_content = remove_non_utf8_characters(doc.page_content)
15     return docs
16
17 def get_num_cpu():
18     return multiprocessing.cpu_count()
19
20 class BaseLoader:
21     def __init__(self) -> None:
22         self.num_processes = get_num_cpu()
23
24     def __call__(self, files: List[str], **kwargs):
25         pass
26
27 class PDFLoader(BaseLoader):
28     def __init__(self) -> None:
29         super().__init__()
30
31     def __call__(self, pdf_files: List[str], **kwargs):
32         num_processes = min(self.num_processes, kwargs["workers"])

```

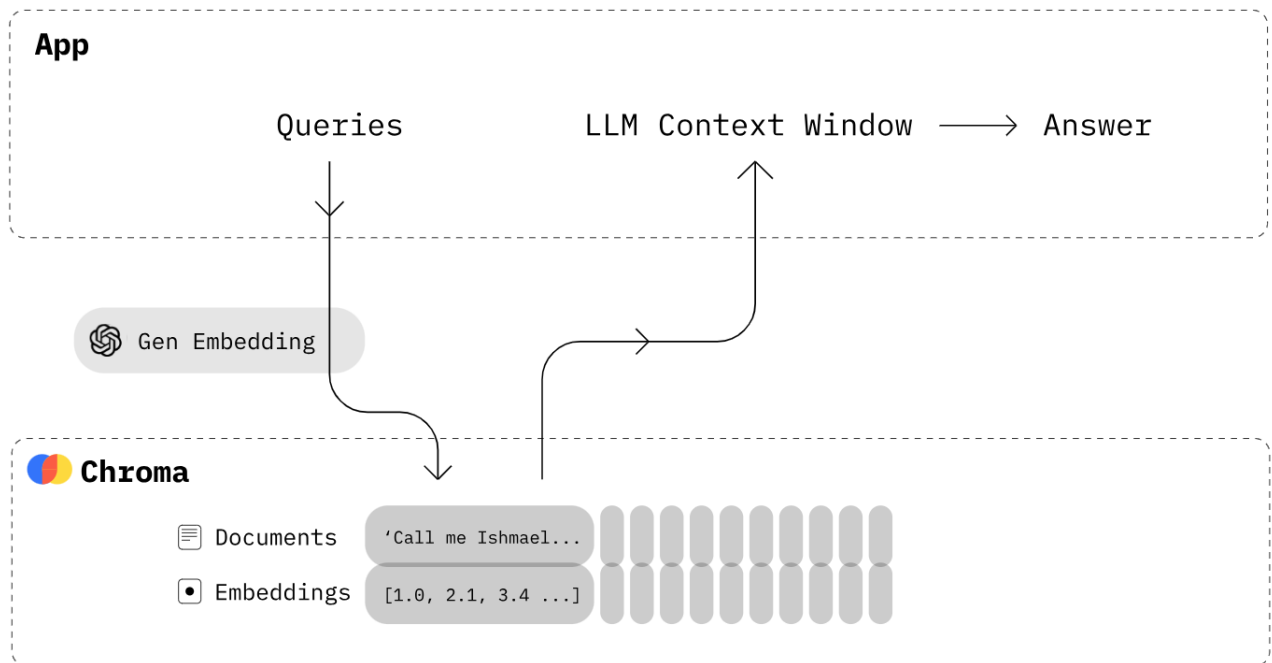
```

33     with multiprocessing.Pool(processes=num_processes) as pool:
34         doc_loaded = []
35         total_files = len(pdf_files)
36         with tqdm(total=total_files, desc="Loading PDFs", unit="file") as pbar:
37             for result in pool.imap_unordered(load_pdf, pdf_files):
38                 doc_loaded.extend(result)
39                 pbar.update(1)
40     return doc_loaded
41
42 class TextSplitter:
43     def __init__(self,
44                 separators: List[str] = ['\n\n', '\n', ' ', ''],
45                 chunk_size: int = 300,
46                 chunk_overlap: int = 0
47                 ) -> None:
48
49         self.splitter = RecursiveCharacterTextSplitter(
50             separators=separators,
51             chunk_size=chunk_size,
52             chunk_overlap=chunk_overlap,
53         )
54     def __call__(self, documents):
55         return self.splitter.split_documents(documents)
56
57 class Loader:
58     def __init__(self,
59                 file_type: str = Literal["pdf"],
60                 split_kwargs: dict = {
61                     "chunk_size": 300,
62                     "chunk_overlap": 0}
63                 ) -> None:
64         assert file_type in ["pdf"], "file_type must be pdf"
65         self.file_type = file_type
66         if file_type == "pdf":
67             self.doc_loader = PDFLoader()
68         else:
69             raise ValueError("file_type must be pdf")
70
71         self.doc_splttter = TextSplitter(**split_kwargs)
72
73     def load(self, pdf_files: Union[str, List[str]], workers: int = 1):
74         if isinstance(pdf_files, str):
75             pdf_files = [pdf_files]
76         doc_loaded = self.doc_loader(pdf_files, workers=workers)
77         doc_split = self.doc_splttter(doc_loaded)
78         return doc_split
79
80     def load_dir(self, dir_path: str, workers: int = 1):
81         if self.file_type == "pdf":
82             files = glob.glob(f"{dir_path}/*.pdf")
83             assert len(files) > 0, f"No {self.file_type} files found in {dir_path}"
84         else:
85             raise ValueError("file_type must be pdf")
86         return self.load(files, workers=workers)

```

II.6. Cập nhật file `src/rag/vectorstore.py`

Tại file này, ta định nghĩa một class để khởi tạo hệ cơ sở dữ liệu vector. Trong project này, chúng ta sẽ sử dụng Chroma. Về việc tìm kiếm tài liệu tương đồng, ta sử dụng FAISS. Như vậy, nội dung của file như sau:



Hình 4: Minh họa việc sử dụng vector database Chroma để truy vấn các tài liệu có liên quan làm context trong prompt. Ảnh: [Link](#).

```

1 from typing import Union
2 from langchain_chroma import Chroma
3 from langchain_community.vectorstores import FAISS
4 from langchain_community.embeddings import HuggingFaceEmbeddings
5
6 class VectorDB:
7     def __init__(self,
8                 documents = None,
9                 vector_db: Union[Chroma, FAISS] = Chroma,
10                embedding = HuggingFaceEmbeddings(),
11                ) -> None:
12
13         self.vector_db = vector_db
14         self.embedding = embedding
15         self.db = self._build_db(documents)
16
17     def _build_db(self, documents):
18         db = self.vector_db.from_documents(documents=documents,
19                                         embedding=self.embedding)
20         return db

```

```

21
22     def get_retriever(self,
23                       search_type: str = "similarity",
24                       search_kwargs: dict = {"k": 10}
25                       ):
26         retriever = self.db.as_retriever(search_type=search_type,
27                                         search_kwargs=search_kwargs)
28     return retriever

```

II.7. Cập nhật file src/rag/offline_rag.py

Tại file này, ta khai báo class `Offline_RAG` để xây dựng một chain về RAG, bao gồm việc sử dụng retriever lấy context, xây dựng prompt và đưa vào model. Nội dung của file như sau:

```

1  import re
2  from langchain import hub
3  from langchain_core.runnables import RunnablePassthrough
4  from langchain_core.output_parsers import StrOutputParser
5
6  class Str_OutputParser(StrOutputParser):
7      def __init__(self) -> None:
8          super().__init__()
9
10     def parse(self, text: str) -> str:
11         return self.extract_answer(text)
12
13     def extract_answer(self,
14                       text_response: str,
15                       pattern: str = r"Answer:\s*(.*)"
16                       ) -> str:
17
18         match = re.search(pattern, text_response, re.DOTALL)
19         if match:
20             answer_text = match.group(1).strip()
21             return answer_text
22         else:
23             return text_response
24
25  class Offline_RAG:
26     def __init__(self, llm) -> None:
27         self.llm = llm
28         self.prompt = hub.pull("rlm/rag-prompt")
29         self.str_parser = Str_OutputParser()
30
31     def get_chain(self, retriever):
32         input_data = {
33             "context": retriever | self.format_docs,
34             "question": RunnablePassthrough()
35         }
36         rag_chain = (
37             input_data
38             | self.prompt
39             | self.llm
40             | self.str_parser

```

```

41         )
42         return rag_chain
43
44     def format_docs(self, docs):
45         return "\n\n".join(doc.page_content for doc in docs)

```

II.8. Cập nhật file src/rag/utils.py

Tại file này, ta khai báo hàm tách phần trả lời của model từ câu prompt (phần bắt đầu từ “Answer:”):

```

1 import re
2
3 def extract_answer(text_response: str,
4                     pattern: str = r"Answer:\s*(.*)"
5                     ) -> str:
6
7     match = re.search(pattern, text_response)
8     if match:
9         answer_text = match.group(1).strip()
10        return answer_text
11    else:
12        return "Answer not found."

```

II.9. Cập nhật file src/rag/main.py

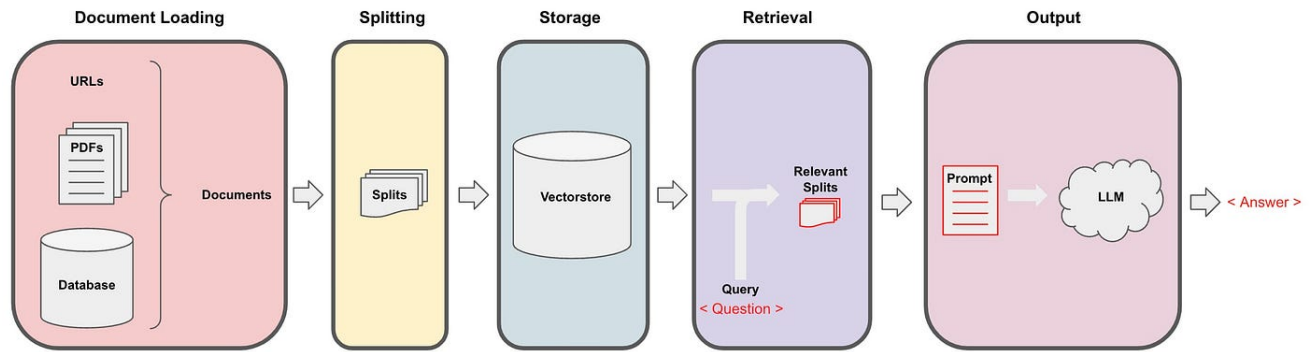
Tại file này, ta khởi tạo toàn bộ các instance của các class, các hàm mà ta đã khai báo trước đó và kết nối chúng vào trong một hàm duy nhất gọi là `build_rag_chain()`:

```

1 from pydantic import BaseModel, Field
2
3 from src.rag.file_loader import Loader
4 from src.rag.vectorstore import VectorDB
5 from src.rag.offline_rag import Offline_RAG
6
7 class InputQA(BaseModel):
8     question: str = Field(..., title="Question to ask the model")
9
10 class OutputQA(BaseModel):
11     answer: str = Field(..., title="Answer from the model")
12
13 def build_rag_chain(llm, data_dir, data_type):
14     doc_loaded = Loader(file_type=data_type).load_dir(data_dir, workers=2)
15     retriever = VectorDB(documents = doc_loaded).get_retriever()
16     rag_chain = Offline_RAG(llm).get_chain(retriever)
17
18     return rag_chain

```

Như vậy, ta đã hoàn thiện toàn bộ các code cần thiết để xây dựng một ứng dụng về RAG. Để tổng quát hóa toàn bộ quy trình, chúng ta có thể tham khảo qua ảnh sau:



Hình 5: Minh họa chuỗi (chain) các bước xây dựng RAG trong LangChain. Ảnh: [Link](#).

II.10. Cập nhật file `src/app.py`

Cuối cùng, ta tạo file dùng để khai báo API với LangServe để triển khai ứng dụng RAG. Đối với LangServe, cách sử dụng gần như tương tự với việc sử dụng FastAPI. Nội dung file code như sau:

```

1 import os
2 os.environ["TOKENIZERS_PARALLELISM"] = "false"
3
4 from fastapi import FastAPI
5 from fastapi.middleware.cors import CORSMiddleware
6
7 from langserve import add_routes
8
9 from src.base.llm_model import get_hf_llm
10 from src.rag.main import build_rag_chain, InputQA, OutputQA
11
12 llm = get_hf_llm(temperature=0.9)
13 genai_docs = "./data_source/generative_ai"
14
15 # ----- Chains-----
16
17 genai_chain = build_rag_chain(llm, data_dir=genai_docs, data_type="pdf")
18
19 # ----- App - FastAPI -----
20
21 app = FastAPI(
22     title="LangChain Server",
23     version="1.0",
24     description="A simple api server using Langchain's Runnable interfaces",
25 )
26
27 app.add_middleware(
28     CORSMiddleware,
29     allow_origins=["*"],
30     allow_credentials=True,
31     allow_methods=["*"],
32     allow_headers=["*"],
33     expose_headers=["*"],

```

```

34 )
35
36 # ----- Routes - FastAPI -----
37
38 @app.get("/check")
39 async def check():
40     return {"status": "ok"}
41
42 @app.post("/generative_ai", response_model=OutputQA)
43 async def generative_ai(inputs: InputQA):
44     answer = genai_chain.invoke(inputs.question)
45     return {"answer": answer}
46
47 # ----- Langserve Routes - Playground -----
48 add_routes(app,
49             genai_chain,
50             playground_type="default",
51             path="/generative_ai")

```

Để khởi động API, chúng ta duy chuyển đến thư mục root của source code trong terminal (trong trường hợp của bài viết sẽ là thư mục `rag_langchain/`), sử dụng lệnh sau (sau khi đã cài đặt các thư viện cần thiết cũng như vector database). Lưu ý, nếu bị lỗi do port đã được sử dụng trong máy của bạn thì có thể thay đổi sang một port khác:

```

1 uvicorn src.app:app --host "0.0.0.0" --port 5000 --reload

```

LangChain Server 1.0 OAS 3.1

/openapi.json

A simple api server using Langchain's Runnable interfaces

generative_ai ⚠ Using pydantic 2.7.0. OpenAPI docs for `invoke`, `batch`, `stream`, `stream_log` endpoints will not be generated. API endpoints and playground should work as expected. If you need to see the docs, you can downgrade to pydantic 1. For example, `pip install pydantic==1.10.13` See <https://github.com/tiangolo/fastapi/issues/10360> for details. ^

GET	/generative_ai/input_schema	Generative AI Input Schema	▼
GET	/generative_ai/output_schema	Generative AI Output Schema	▼
GET	/generative_ai/config_schema	Generative AI Config Schema	▼

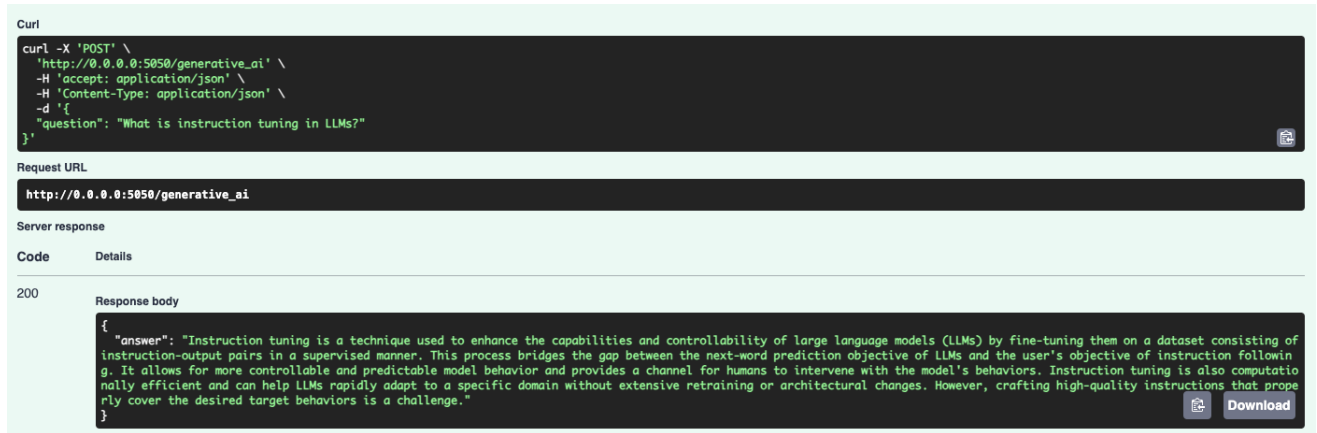
generative_ai/config Endpoints with a default configuration set by `config_hash` path parameter. Used in conjunction with share links generated using the LangServe UI playground. The hash is an LZString compressed JSON string. ^

GET	/generative_ai/c/{config_hash}/input_schema	Generative AI Input Schema With Config	▼
GET	/generative_ai/c/{config_hash}/output_schema	Generative AI Output Schema With Config	▼
GET	/generative_ai/c/{config_hash}/config_schema	Generative AI Config Schema With Config	▼

default

GET	/check	Check	▼
POST	/generative_ai	Generative AI	▼
POST	/generative_ai/token_feedback	Create Feedback From Token	▼

Hình 6: Minh họa API sau khi ta triển khai thành công.



```
Curl
curl -X 'POST' \
  'http://0.0.0.0:5050/generative_ai' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "question": "What is instruction tuning in LLMs?"
  }'
```

Request URL

http://0.0.0.0:5050/generative_ai

Server response

Code	Details
200	Response body <pre>{ "answer": "Instruction tuning is a technique used to enhance the capabilities and controllability of large language models (LLMs) by fine-tuning them on a dataset consisting of instruction-output pairs in a supervised manner. This process bridges the gap between the next-word prediction objective of LLMs and the user's objective of instruction following. It allows for more controllable and predictable model behavior and provides a channel for humans to intervene with the model's behaviors. Instruction tuning is also computationally efficient and can help LLMs rapidly adapt to a specific domain without extensive retraining or architectural changes. However, crafting high-quality instructions that properly cover the desired target behaviors is a challenge." }</pre> Download

Hình 7: Minh họa một kết quả của model thông qua API mà chúng ta đã xây dựng.

III. Câu hỏi trắc nghiệm

- LangChain được sử dụng nhằm mục đích gì?
 - Web Scraping.
 - Model Quantization.
 - Building language model-powered applications.
 - Database Management.
- Nội dung nào dưới đây là một thành phần cốt lõi của LangChain?
 - Transformers.
 - Agents.
 - Callbacks.
 - Hooks.
- Trong LangChain, mục đích trong việc sử dụng PromptTemplate là?
 - Tạo các trường thông tin trong hệ cơ sở dữ liệu lưu thông tin người dùng.
 - Định nghĩa các tính năng trong giao diện của người dùng.
 - Tối ưu tốc độ xử lý của mô hình.
 - Chuẩn hóa một cấu trúc phản hồi nhất quán từ mô hình.
- Xét đoạn code dưới đây:

```
1 from langchain_openai import ChatOpenAI
2 from langchain_openai import OpenAI
3
4 llm = OpenAI()
5 chat_model = ChatOpenAI(model="gpt-3.5-turbo-0125")
```

- Ý nghĩa của đoạn code trên là?
- Khởi tạo model GPT 3.5 Turbo-0125.
 - Tải pre-trained model GPT 3.5 Turbo-0125.
 - Kiểm tra tốc độ đường truyền với ChatGPT API.
 - Các đáp án trên đều sai.
- Ý nghĩa của phương thức `from_template()` trong class `PromptTemplate` là?
 - Để khởi tạo prompt template từ một file.
 - Để khởi tạo prompt template từ một string.
 - Để khởi tạo prompt template từ một danh sách các tin nhắn.
 - Để khởi tạo prompt template từ một prompt template có sẵn.

6. Trong LangChain, loại OutputParser nào dưới đây có thể được sử dụng để trả về kết quả của mô hình dưới dạng JSON?
- (a) PydanticOutputParser.
 - (b) RegexOutputParser.
 - (c) JsonOutputParser.
 - (d) YamlOutputParser.

7. Xét đoạn code dưới đây:

```
1 from langchain import HuggingFaceHub
2 from langchain import PromptTemplate
3
4 template = """Question: {question}
5
6 Answer: """
7 prompt = PromptTemplate(
8     template=template,
9     input_variables=['question']
10 )
11
12 hub_llm = HuggingFaceHub(
13     repo_id="google/flan-t5-xl"
14 )
15
16 llm_chain = prompt | hub_llm
17
18 print(llm_chain.run("What year was the World Cup first held?"))
```

Ý nghĩa của các dòng code 16 là gì?

- (a) Khai báo hệ cơ sở dữ liệu vector.
 - (b) Khởi tạo LLMChain với LLM và Prompt.
 - (c) Cài đặt ủy quyền và bảo mật cho người dùng.
 - (d) Phân tích và trực quan hóa dữ liệu.
8. Xét đoạn code dưới đây:

```
1 from langchain_community.document_loaders import PyPDFLoader
2
3 pdf_loader = PyPDFLoader(url, extract_images=True)
4
5 docs = pdf_loader.load()
```

Tham số `extract_images` tại dòng code 3 có chức năng gì?

- (a) Trả về tất cả ảnh từ file pdf.
- (b) Bỏ qua ảnh, chỉ load text.
- (c) Phân tích ảnh thành vector.
- (d) Chuyển đổi ảnh trong file pdf thành text.

9. Tại sao chúng ta cần phải chia nhỏ các tài liệu đầu vào thành các tài liệu ngắn hơn? Chọn câu trả lời **SAI**.

- (a) Giúp LLM tập trung tạo ra câu trả lời chỉ dựa trên các thông tin có liên quan.
- (b) Tiết kiệm bộ nhớ cho phần cứng.
- (c) Chỉ dựa vào một phần nhỏ tài liệu thì mô hình vẫn trả lời chính xác.
- (d) Giúp mô hình LLM chạy nhanh hơn.

10. Xét đoạn code dưới đây:

```
1 from langchain_community.document_loaders import PyPDFLoader
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3 from langchain_community.embeddings import HuggingFaceEmbeddings
4 from langchain_chroma import Chroma
5
6 pdf_url = "https://arxiv.org/pdf/2401.18059v1.pdf"
7
8 # PDF loader
9 pdf_loader = PyPDFLoader(pdf_url, extract_images=True)
10 pdf_pages = pdf_loader.load()
11
12 # Splitter
13 splitter = RecursiveCharacterTextSplitter(
14     chunk_size=300,
15     chunk_overlap=0,
16 )
17 docs = splitter.split_documents(pdf_pages)
18
19 # Embedding model
20 embedding_model = HuggingFaceEmbeddings()
21
22 # vector store
23 chroma_db = Chroma.from_documents(docs, embedding=embedding_model)
```

Nhiệm vụ của `embedding_model` là gì?

- (a) Dùng biến đổi chuỗi đầu vào thành các vector cho cơ sở dữ liệu vector.
- (b) Dùng để lập chỉ mục cho cơ sở dữ liệu.
- (c) Dùng để tìm kiếm tài liệu.
- (d) Dùng để tính toán độ tương đồng.

IV. Phụ lục

1. **Datasets:** Các file dataset được đề cập trong bài có thể được tải tại [đây](#).
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại [đây](#) (**Lưu ý:** Sáng thứ 3 khi hết deadline phần nội dung này, ad mới copy các tài liệu bài giải nêu trên vào đường dẫn).
3. **Demo:** Web demo của ứng dụng có thể được truy cập tại [đây](#).
4. **Rubric:**

Mục	Kiến Thức	Đánh Giá
I.	- Kiến thức về mô hình ngôn ngữ lớn (LLMs). - Kiến thức bài toán Retrieval Augmented Generation (RAG).	- Hiểu được các nội dung cơ bản về LLMs và RAG. Input Output của bài toán RAG và luồng xử lý cơ bản.
II.	- Các kiến thức cơ bản về thư viện LangChain. - Tổng quan các cách bước cơ bản trong việc sử dụng LangChain. - Chức năng một số hàm cơ bản trong LangChain nhằm phục vụ cho cài đặt ứng dụng RAG. - Khái niệm về API và luồng triển khai API cơ bản.	- Nắm được các nội dung và chức năng cơ bản của thư viện LangChain. - Có thể sử dụng thư viện LangChain để cài đặt một ứng dụng RAG sử dụng LLMs.

- Hết -